

Lab 06 Loop Statements II

Objective

The purpose of this Lab. is to familiarize student with how to write programs that repeatedly execute one or more statements with the use of repetition control structure (while, do .. while, for).

Current Lab Learning Outcomes (LLO)

Student will learn how to write Java programs using loop statements, (while, do .. while, for), to repeat Java statements more than times. Students will practice several kinds of problems using loop statements. There are some exercises, through which they will understand the concept learn in this chapter.

Lab Requirements

NetBeans

Lab Description

Theory Review (5 Minutes):

- Java provides a powerful construct called a loop. A loop controls how many times an operation (or a sequence of operations) is performed.
- Java provides three types of loop statements (while loops, do-while loops and for loops).

The **do-while** Loop

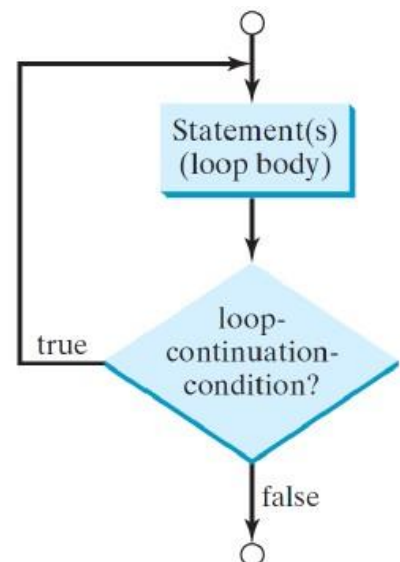
☞ The **do-while** loop is very similar to the while loop

☞ Difference:

- The **do-while** loop executes the loop body first
- Then it checks the loop continuation condition

☞ Syntax:

```
do {  
    // loop body  
    Statement(s);  
} while (loop-continuation-condition);
```



Example: Print "Welcome to Java" one hundred times using do..while statement

```
int count = 0;
do{
    System.out.println("Welcome to Java");
    count++;
}while(count < 100);
```

The for Loop

Often you will use a **while** loop to iterate a certain number of times

Example:

```
i = initialValue; // Initialize loop control variable
while (i < endValue) {
    // Loop body
    Statement(s);
    i++; // Adjust loop control variable
}
```

– If you want to iterate a specific number of times, it is better/easier to just use a **for** loop!

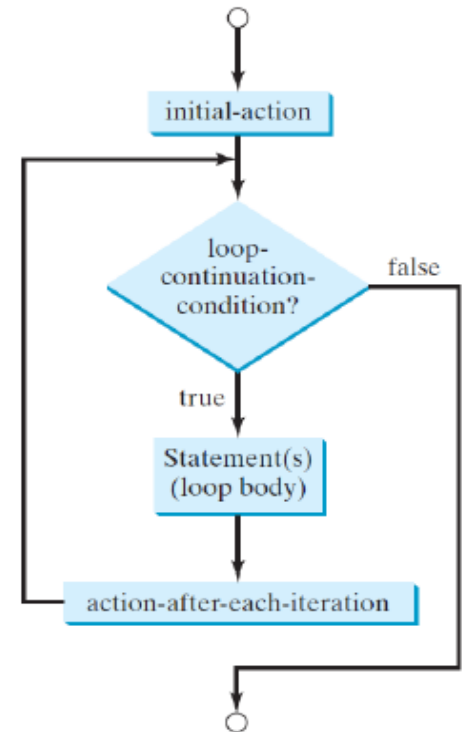
The for Loop

A **for** loop can simplify the **while** loop on the previous page:

```
for (i = initialValue; i < endValue; i++) {
    // Loop body
    Statement(s);
}
```

In general, the syntax of a **for** loop is:

```
for (initial-action; loop-condition; action-after) {
    // Loop body
    Statement(s);
}
```



Example: Print "Welcome to Java" one hundred times using for statement

```
for(int count = 0; count < 100; count++)
{
    System.out.println("Welcome to Java");
}
```

Practice Activity with Lab Instructor (10 minutes)

(Printing Series) Write a program that displays numbers for a given number, **n** to **1** using do .. while statement.

Solution:

1. Open NetBeans and create a new project
2. Create a new java main class and write its name as **PrintSeriesInReverse**
3. Write the following code inside the main method

```
1
2 import java.util.Scanner;
3
4 public class PrintSeriesInReverse {
5     public static void main(String[] args) {
6         Scanner input =new Scanner(System.in);
7         System.out.print("Enter end of series: ");
8         int n=input.nextInt();
9         int count=n;
10        do
11        {
12            System.out.println(count);
13            count--;
14        }while(count>=1);
15    }
16
17 }
18
```

Output - ProgramsForLabManuals (run) ×

```
run:
Enter end of series: 10
10
9
8
7
6
5
4
3
2
1
BUILD SUCCESSFUL (total time: 2 seconds)
```

Short Exercise (5 minutes)

Rewrite the above program using **for** statement.

Lab Assessment

Individual Activities: (60 minutes)

1. **(Reverse a string)** Write a program that prompts the user to enter a string and displays the string in reverse order.

```
Enter a string: ABCD ↵ Enter
The reversed string is DCBA
```

2. **(Perfect number)** A positive integer is called a perfect number if it is equal to the sum of all of its positive divisors, excluding itself. For example, 6 is the first perfect number because $6 = 3 + 2 + 1$. The next is $28 = 14 + 7 + 4 + 2 + 1$. There are four perfect numbers less than 10,000. Write a program to find all these four numbers.
3. **(Longest common prefix)** Write a program that prompts the user to enter two strings and displays the largest common prefix of the two strings. Here are some sample runs:

```
Enter the first string: Welcome to C++ ↵ Enter
Enter the second string: Welcome to programming ↵ Enter
The common prefix is Welcome to
```

```
Enter the first string: Atlanta ↵ Enter
Enter the second string: Macon ↵ Enter
Atlanta and Macon have no common prefix
```

4. **(Find the highest score)** Write a program that prompts the user to enter the number of students and each student's name and score, and finally displays the name of the student with the highest score.
5. **(Sum a series)** Write a program to sum the following series:

$$\frac{1}{3} + \frac{3}{5} + \frac{5}{7} + \frac{7}{9} + \frac{9}{11} + \frac{11}{13} + \cdots + \frac{95}{97} + \frac{97}{99}$$

NESTED LOOP STATEMENTS

Objective

The objective of this Lab is to familiarize student with how to write programs that repeatedly execute one or more statements with the use of repetition control structure (while, do .. while, for) as a nested loop.

Current Lab Learning Outcomes (LLO)

By completion of the lab the students should be able to

1. write Java programs using loop statements, (**while**, **do .. while**, **for**), as a nested loop to repeat Java statements more than times.
2. use **break** and **continue** keywords. **break** allows you to immediately terminate a loop whereas **continue** ends the current iteration of the loop.
3. do practice several kinds of problems using nested loops.

There are some exercises, through which they will understand the concept learn in this chapter

Lab Requirements

NetBeans IDE.

Lab Description

Nested Loops:

- A loop can be nested inside another loop.
- Nested loops consist of an outer (outside) loop and one or more inner loops

Example:

```
for (int i = 0; i < 100; i++) {  
    for (int j = 0; j < 100; j++) {  
        // some code here  
    }  
}
```

Keywords break and continue

1. **break** and **continue** provide additional control inside the loops
2. **break** allows you to immediately terminate a loop.
3. **continue** ends the current iteration of the loop and program control skips until the end of the loop body

```
1 public class TestBreak {
2     public static void main(String[] args) {
3         int sum = 0;
4         int number = 0;
5
6         while (number < 20) {
7             number++;
8             sum += number;
9             if (sum >= 100)
10                break;
11        }
12
13        System.out.println("The number is " + number);
14        System.out.println("The sum is " + sum);
15    }
16 }
```

The number is 14
The sum is 105

```
1 public class TestContinue {
2     public static void main(String[] args) {
3         int sum = 0;
4         int number = 0;
5
6         while (number < 20) {
7             number++;
8             if (number == 10 || number == 11)
9                continue;
10            sum += number;
11        }
12
13        System.out.println("The sum is " + sum);
14    }
15 }
```

The sum is 189

Lab Assessment

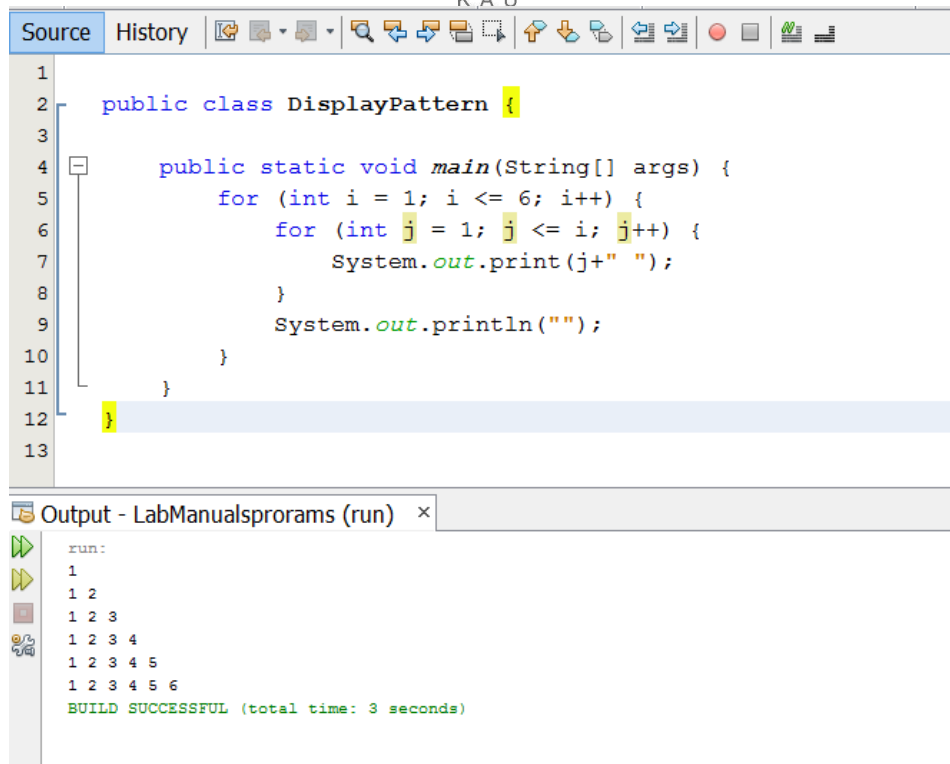
1. Practice Activity with Lab Instructor (10 minutes)

(Display pattern using loops) Use nested loops that display the following patterns.

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
```

Solution:

1. Open NetBeans and create a new project
2. Create a new java main class and write its name as **DisplayPattern**
3. Write the following code inside the main method



```

1
2 public class DisplayPattern {
3
4     public static void main(String[] args) {
5         for (int i = 1; i <= 6; i++) {
6             for (int j = 1; j <= i; j++) {
7                 System.out.print(j+" ");
8             }
9             System.out.println("");
10        }
11    }
12 }
13

```

Output - LabManualsprograms (run)

```

run:
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
BUILD SUCCESSFUL (total time: 3 seconds)

```

1. Short Exercise (5 minutes)

Rewrite the above program to display the following pattern.

```

1 2 3 4 5 6
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1

```

Solution:

```

public class DisplayPattern2 {

    public static void main(String[] args) {
        System.out.print("Enter the number of lines:");
        Scanner input = new Scanner(System.in);
        // get the total number of lines n.
        int n = input.nextInt();
        // Loop through the lines from 1 to n (i=0 to n-1
        for (int i = 0; i < n; i++) {

            // Printing number incrementally from 1 to line number j
            for (int j = 1; j <= n - i; j++) {
                System.out.print(j + " ");
            }
            System.out.println();
        }
    }
}

```

Lab Assessment

Three Problems\ exercises should be completed **in 60 minutes individually**

1. **(Display two patterns using loops)** Use nested loops that display the following patterns in two separate programs:

```

        1          1 2 3 4 5 6
      2 1          1 2 3 4 5
    3 2 1          1 2 3 4
  4 3 2 1          1 2 3
5 4 3 2 1          1 2
6 5 4 3 2 1          1
```

2. **(Perfect number)** A positive integer is called a perfect number if it is equal to the sum of all of its positive divisors, excluding itself. For example, 6 is the first perfect number because $6 = 3 + 2 + 1$. The next is $28 = 14 + 7 + 4 + 2 + 1$. There are four perfect numbers less than 10,000. Write a program to find all these four numbers.
3. **(Game: scissor, rock, paper)** Program exercise in Lab 5 gives a program that plays the scissor-rock- paper game. Revise the program to let the user continuously play until either the user or the computer wins more than two times than its opponent.
4. **(Simulation: heads or tails)** Write a program that simulates flipping a coin one million times and displays the number of heads and tails.
5. **(Display numbers in a pyramid pattern)** Write a nested for loop that prints the following output:

```

          1
        1 2 1
      1 2 4 2 1
    1 2 4 8 4 2 1
  1 2 4 8 16 8 4 2 1
1 2 4 8 16 32 16 8 4 2 1
1 2 4 8 16 32 64 32 16 8 4 2 1
1 2 4 8 16 32 64 128 64 32 16 8 4 2 1
```
